Object Oriented Design

Overview

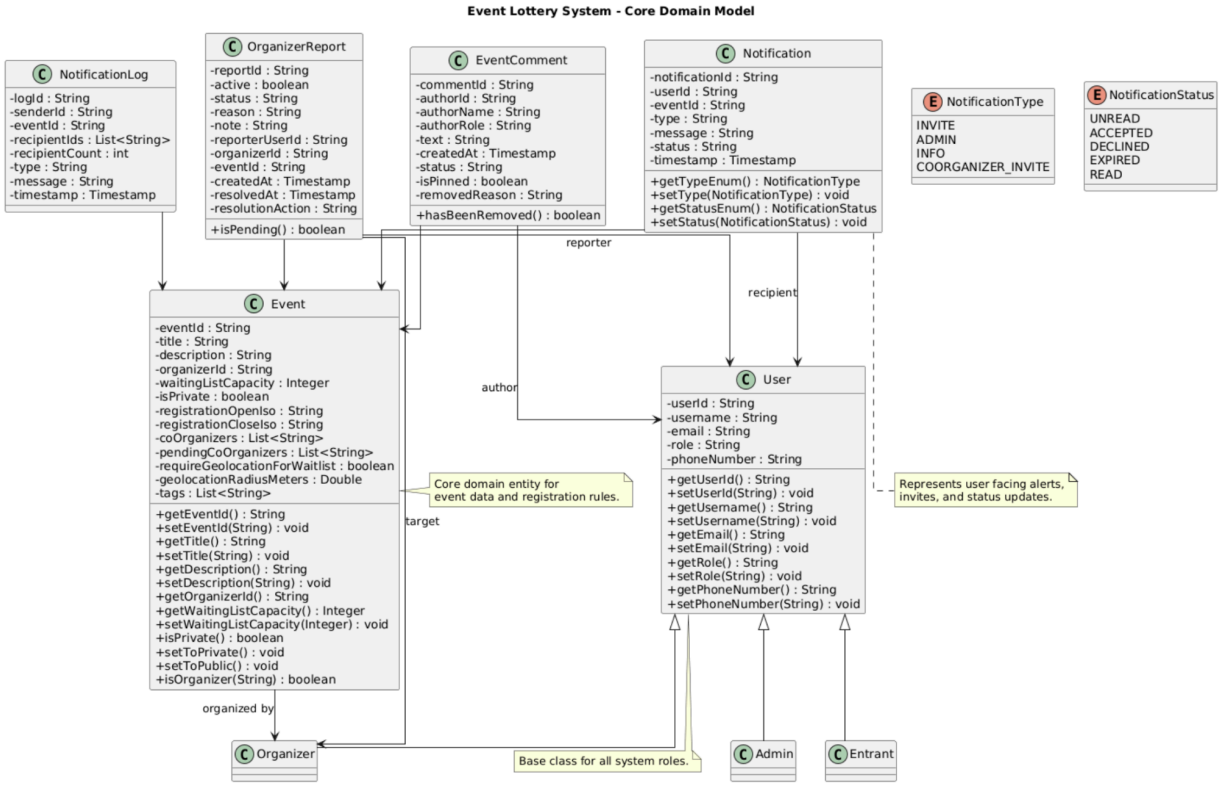
The Event Lottery System is built using an object oriented design that breaks the system down into three main layers: core domain entities, repositories and business logic, and user interface components. This layered structure makes it easier to maintain, grow, and separate concerns.

To make it easier to read, the design is shown with several UML class diagrams, each of which shows a different part of the system. Also, there is a full UML diagram that shows all of the main classes and their relationships, which gives a full picture of the system's architecture.

UML Diagrams

Core Domain Model

Figure 1: Core Domain Model UML Diagram



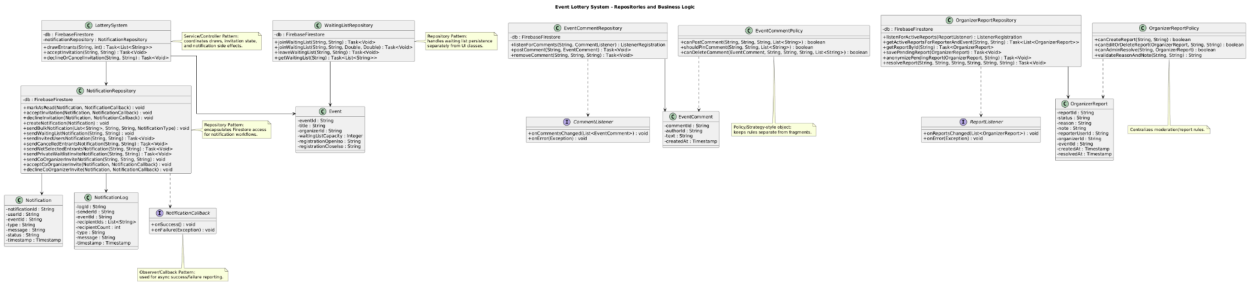
This diagram represents the main data entities of the system:

- User is the base class representing all users
- Admin, Entrant, and Organizer inherit from User
- Event represents event data and is associated with an Organizer
- Notification represents messages sent to users
- NotificationLog tracks sent notifications
- EventComment represents user comments
- OrganizerReport represents reports submitted against organizers

This design makes sure that entities are well-defined and that their relationships are clear.

Repositories and Business Logic

Figure 2: Repository and Logic UML Diagram



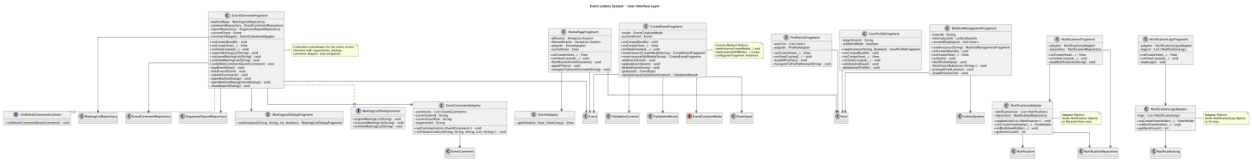
This diagram shows the system's control layer:

- NotificationRepository
- WaitingListRepository
- EventCommentRepository
- OrganizerReportRepository
- LotterySystem

These classes use the Repository pattern to separate data access from UI logic and hide database interactions. LotterySystem is a service/controller class that handles lottery draws, invitations, and notification updates.

User Interface Layer

Figure 3: User Interface UML Diagram



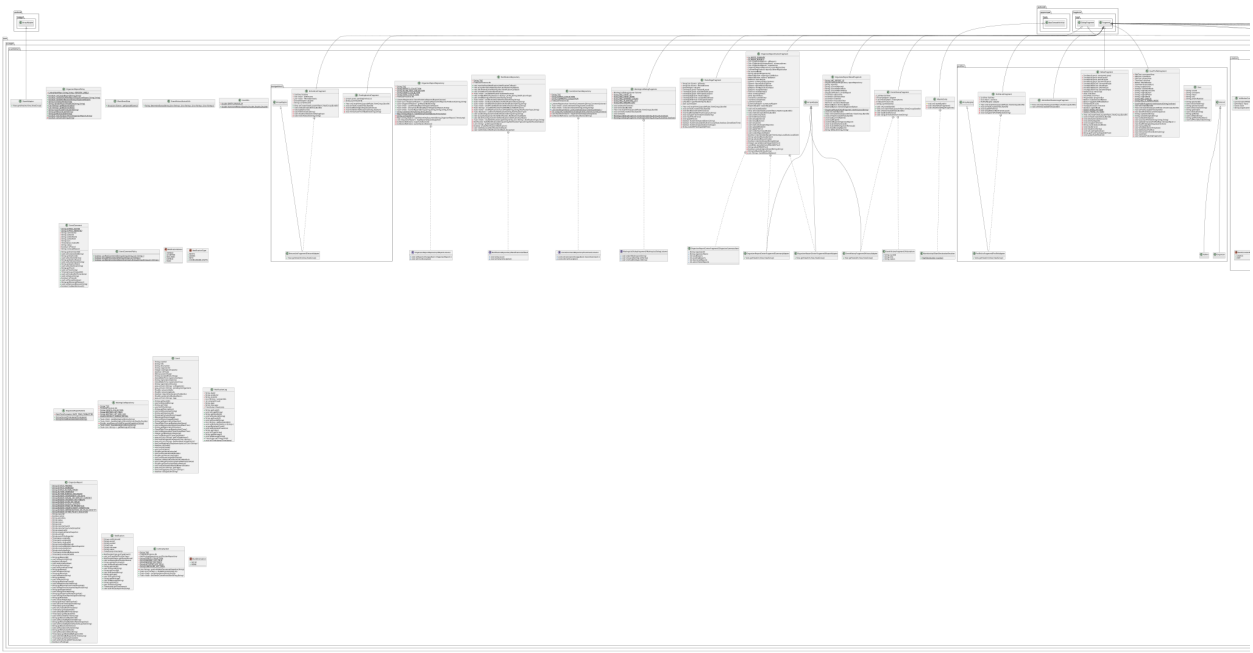
This diagram shows the UI structure using fragments and adapters:

- Fragments:
    - HomePageFragment
    - CreateEventFragment
    - EventOverviewFragment
    - NotificationsFragment
    - WaitlistManagementFragment
- Adapters:
    - NotificationsAdapter
    - NotificationLogsAdapter
    - EventAdapter

Adapters use the Adapter pattern to turn domain objects into UI components that can be shown. EventOverviewFragment acts like a controller that keeps track of user actions, repository calls, and updates to the UI.

Complete System Diagram

Figure 4: Complete UML Class Diagram



Along with the simplified diagrams, there is also a full UML diagram that shows all the main classes and relationships in one place. This diagram shows the whole system architecture, including extra supporting classes and methods that are left out of the simpler diagrams to make things clearer.

Design Patterns

The system utilizes various object oriented design patterns:

Repository Pattern

Used in:

- NotificationRepository
- WaitingListRepository
- EventCommentRepository
- OrganizerReportRepository

Handles database operations and separates persistence logic from the user interface.

Adapter Pattern

Used in:

- NotificationsAdapter
- NotificationLogsAdapter
- EventAdapter
- EventCommentAdapter

Converts domain objects into formats suitable for the user interface.

Observer / Callback Pattern

Used in:

- NotificationCallback
- CommentListener
- ReportListener

Supports asynchronous operations and real time updates.

Factory Method Pattern

Used in:

- CreateEventFragment.newInstance(...)
- ValidationResult.valid(...) and invalid(...)

Manages object creation and ensures proper initialization.

Policy / Strategy Pattern

Used in:

- EventCommentPolicy
- OrganizerReportPolicy

Handles business rules while isolating them from UI logic.

Service / Controller Pattern

Used in:

- LotterySystem
- EventOverviewFragment

Coordinates system behavior and manages interactions between components.

Design Rationale

The design emphasizes:

- **Separation of concerns:** UI, logic, and data access have a clear distinction.
- **Scalability:** New features can be added without modifying existing components.
- **Maintainability:** Centralized logic in repositories and policy classes helps minimize duplication.
- **Reusability:** Core models and repositories are reused across multiple parts of the system.

Note

To keep the diagrams easy to read, they focus on the main classes, attributes, and relationships. Lower level details, such as UI elements (e.g. buttons, text fields), helper utilities and internal adapter components are left out, unless they are essential to understanding the design.